D06 · CLI 화면 설계서

promptbench

터미널 출력 장면 10개 — 승인 화면 · 예산 정지 · 오류 문구

문서 버전	1.0
작성일	2026-10-20
작성자	김태민
문서 상태	구현·실측 완료본

변경 이력

버전	일자	내용	작성자
1.0	2026-10-20	최초 작성	김태민

1. 장면 목록

장면 10개, 모두 터미널 출력이다. 연결 명령은 D05 명령 일람(`commands.yaml` 의 scenes), 관련 요구사항은 D02 의 장면 열에서 역으로 모았다. 각 장면 쪽의 요소 번호는 `SN-03-4` 처럼 장면 ID 에 붙여 부른다.

흐름 — 준비(SN-01) → 계획(SN-02, 예산에 안 맞으면 SN-04) → 승인(SN-03) → 실행(SN-05, 상한 직전이면 SN-06 정지, SN-07 이어 하기) → 조회(SN-10) → 채점 보정(SN-08) → 보고(SN-09). 장면의 금액은 하나의 예시 계획(R1)에서 계산했지만, SN-04 · SN-06 은 남은 예산이 적은 경우를 보이려는 가정이라 장부 값이 서로 이어지지 않는다.

> **범위 밖** — 웹·앱 화면은 없다(CLI 만). `pilot` · `judge-R1` · `judge-R2` · `latency` 는 명령이 아니라 plan 의 stage 이름이라 전용 장면 없이 SN-02 · SN-03 의 stage 줄로 나온다. 모델 호출 방식 중 effort · structured outputs · thinking 토큰 집계는 구현 1단계에서 **확인함**(D08 6장)이며 해당 요소 설명에 적었다.

ID	장면	연결 명령	관련 요구사항	요소
SN-01	**prepare 결과** 데이터를 내려받고 고정 시드로 표본을 나눈 뒤 표본 ID 만 기록한다. 원본은 저장소에 넣지 않는다.	CMD-01 prepare	REQ-001 · REQ-002 · REQ-003 · REQ-004 · REQ-005 · REQ-006 · REQ-007 · REQ-008	8
SN-02	**plan 결과(배분표·금액 3종)** R1 계획 예시. 금액은 빌드할 때 가격표(pricing.yaml)로 계산한 값이며 토큰 수는 가정이다. 아래쪽은 run --retry-failed 가 만든 새 계획이다.	CMD-02 plan · CMD-04 run	REQ-004 · REQ-009 · REQ-010 · REQ-011 · REQ-012 · REQ-013 · REQ-014 · REQ-015 · REQ-017 · REQ-018 · REQ-019 · REQ-020 · REQ-042 · REQ-059	11
SN-03	**approve 승인 화면** 승인 화면 예시. 금액은 pricing.yaml 로 계산한 예시 값이다. 가격표 확인일 경고와 latency 줄은 해당 조건에서만 나온다.	CMD-03 approve	REQ-021 · REQ-022 · REQ-023 · REQ-024 · REQ-025 · REQ-026 · REQ-031 · REQ-055 · REQ-057	9
SN-04	**예산 초과로 계획 생성 실패** 줄일 수 있는 것을 모두 줄여도 남은 예산을 넘는 경우. 앞선 지출이 커서 남은 예산이 적다고 가정한 예시다.	CMD-02 plan	REQ-016 · REQ-054	4
SN-05	**run 진행 표시** 승인된 계획을 작은 배치로 나눠 순서대로 낸다. 배치가 끝날 때마다 실제 비용을 장부에 더한다. 실제 비용은 평균 출력 토큰이 가정의 1.1배라고 가정한 예시다.	CMD-04 run	REQ-018 · REQ-020 · REQ-027 · REQ-028 · REQ-032 · REQ-033 · REQ-034 · REQ-035 · REQ-036 · REQ-037	8
SN-06	**run 상한 정지(halted)** 다음 배치의 보수 금액이 남은 예산보다 크면 제출 전에 멈춘다. 지출은 상한 직전이라고 가정한 예시이며 SN-05 의 장부를 이어 가지 않는다.	CMD-04 run	REQ-029	5
SN-07	**run --resume** 중단된 계획을 같은 해시로 잇는다. 위쪽은 재개 성공, 아래쪽은 남은 예산이 모자라 halted 로 남는 경우다.	CMD-04 run	REQ-030 · REQ-035	6
SN-08	**calibrate 채점 화면** 사람이 교정 세트 30개를 같은 루브릭으로 채점한다. 요약문과 교차표는 가상이며 일치율과 κ 는 교차표에서 계산한 값이다.	CMD-06 calibrate	REQ-043 · REQ-044	7
SN-09	**report 요약 출력** report 가 터미널에 요약하는 출력. 결과표는 D03 의 가상 예시(고정 시드 30건)를 같은 함수로 다시 실행한 값이며 실측이 아니다. Sonnet · Opus 행은 R1 이 아니라 R2(Sonnet low/medium · Opus low × 공통 30건, 이긴 프롬프트)에서 온 것으로 가정한다.	CMD-07 report	REQ-039 · REQ-040 · REQ-041 · REQ-044 · REQ-045 · REQ-046 · REQ-047 · REQ-048 · REQ-049 · REQ-050 · REQ-051 · REQ-052 · REQ-053	8
SN-10	**status 장부** status 는 장부와 진행 상태를 읽기만 한다. 값은 SN-05 · SN-07 과 같은 가정에서 계산한 예시다.	CMD-05 status	REQ-028 · REQ-038 · REQ-054	6

2. 공통 가이드

2-1. 문구 원칙

오류는 세 줄	**무엇이**(첫 줄, ✗ 또는 ⚠ 로 시작) / **왜**(이유:) / **다음에 할 일**(다음: , 그대로 칠 수 있는 명령). 세 줄 중 하나라도 빠지면 사용자는 로그를 뒤지게 된다. 거부로 끝나면 부작용(계획 파일 · 승인 · 과금 호출)이 없었다는 것도 한 줄 덧붙인다(테스트로 형식을 고정).
금액은 항상 근거와 함께	금액 줄에는 어떤 금액인지(예상 · 보수 · 최악)와 기준(출력 평균 · p90 · max_tokens)을 같이 적는다. 승인 · 통과의 기준은 보수 금액이다. 이 문서의 금액은 모두 **예시**이고 단가는 가격표로 계산한다.
상태 이름은 그대로	계획 상태 5종(draft · approved · running · done · halted)은 `states.yaml` 영문 이름 그대로 쓴다. 번역하면 status · DB · 문서 사이에서 같은 것을 다르게 부르게 된다. 뜻은 옆에 한국어로 붙인다.
승인은 해시에 묶인다	승인 화면 · run · resume 은 해시 앞 8자를 같이 보인다. 해시가 같으면 같은 계획이고(`--resume` 은 새 승인 없음), 다르면 새 계획이다(`--retry-failed` · 줄인 계획은 다시 승인).
멈추는 건 제출 전에	상한 점검은 배치를 내기 전에 한다. 정지 화면에는 누적 · 남은 예산 · 다음 배치 보수 금액을 같은 표에 두고, 아직 내지 않은 요청은 pending 으로 남았다고 알린다.
실패를 숨기지 않는다	후보 쪽 실패는 오답으로 세고 유형별 건수를 보인다. T3 의 채점 불가는 별도로 센다. 0건일 때도 칸은 사라지지 않는다 — 없음과 고장을 구분할 수 없기 때문이다.
표는 HTML 표로	한글은 등폭 글꼴에서도 두 칸을 차지해 세로줄 · 가로줄 상자 문자로 그린 표가 어긋난다. 터미널 목업의 표는 `<table class="term">` 로 그리고 문자 그림을 쓰지 않는다(테스트로 고정).
읽기와 쓰기를 구분	status · calibrate · report 는 과금도 승인도 없다. 과금 호출은 run 만 하고, 그 앞에 항상 plan → approve 가 있다.

2-2. 오류 문구 예 (POL-21)

```
✗ 계획을 만들지 않았습니다 — 보수 금액이 남은 예산을 넘습니다
  이유: 줄일 수 있는 것을 모두 줄여도 공통 30건만으로 넘습니다
  다음: promptbench status 로 장부를 확인하세요
과금 호출 0건 · 종료코드는 실패(0 아님)
```

2-3. 색 의미 — 계획 상태

글자색은 상태 이름에 겹쳐 쓴다. 색만으로 상태를 구분하지 않는다(흑백 인쇄 · 색약) — 이름을 항상 같이 쓴다. 굵게 = 지금 진행 중이거나 사람이 확인해야 하는 상태.

ID	이름 · 색	뜻 (states.yaml)
ST-P-01	**draft**	계획 파일이 만들어졌지만 아직 승인되지 않은 상태
ST-P-02	**approved**	계획 해시에 승인이 기록된 상태
ST-P-03	**running**	배치를 작게 나눠 순서대로 제출하며 배치마다 실제 비용을 장부에 더하는 상태
ST-P-04	**done**	모든 배치가 끝난 상태
ST-P-05	**halted**	상한 직전에 멈추거나 프로세스가 중단된 상태

2-4. 기호

✓	성공 · 통과
✗	실패 · 거부 — 계획이나 실행이 막힘
⚠	경고 — 멈췄거나 확인이 필요하지만 막지는 않음
[y/N]	확인 입력 — 대문자가 기본값
—	값 없음(아직 일어나지 않음)

2-5. 금액 표기

금액은 `$0.0000` — 달러 기호, 소수 넷째 자리까지, 천 단위 쉼표 없음. 요청 한 건이 1센트도 안 되기 때문에 둘째 자리로는 차이가 사라진다. 상한 · 합계 · 배치 · 장부가 모두 같은 자릿수다. 이 문서의 금액은 빌드할 때 가격표(`config/pricing.yaml`)와 `request_cost` 로 계산한 **예시**이며 토큰 수는 가정이다(실측은 D09). 건수는 천 단위 쉼표를 쓴다.

```
예상 금액      $0.7725
보수 금액      $1.1850
최악 금액      $4.2760
상한           $5.0000
```

2-6. D06 설계 결정

앞 문서(D02 · D03 · D05)에 근거가 없어 이 문서에서 정한 것이다. 구현 때 바뀔 수 있다.

채점자 판정 숨김 (SN-08)	사람이 채점자 답을 보고 따라가면 일치율이 부풀려진다.
종료코드 2 (SN-06)	상한 정지는 일반 오류(0 아님)와 구분되는 정상적 중단이라 따로 둔다. 계획 생성 실패(SN-04)는 REQ-016 의 "종료 코드가 실패"만 따른다.
q 로 저장 후 중단 (SN-08)	30개 채점 도중 멈춰도 입력이 사라지지 않게 한다.

계획 ID 모양 (plan-r1-001 등)	사람이 입력하는 값이라 stage 가 보이는 짧은 이름이 낫다. 실제 형식은 구현에서 정한다.
재개 거부 화면 (SN-07)	POL-08 의 재개 조건 미달을 사용자가 만나는 모습을 보이려 만든 화면이다.

2-7. 프레임

터미널(폭 760px · 어두운 배경 · 등폭 글꼴 D2Coding → Menlo)이다. 목업 왼쪽의 주황 원 번호가 오른쪽 설명 번호다. 모든 목업 아래에 **예시 화면 — 수치는 가상** 표시를 단다.

SN-01 prepare 결과 CLI · 터미널 promptbench



예시 화면 — 수치는 가상 데이터를 내려받고 고정 시드로 표본을 나눈 뒤 표본 ID 만 기록한다. 원본은 저장소에 넣지 않는다.

1 명령 입력 SN-01-1

첫 단계다. 앞서 할 일이 없고 과금도 없다. 이후 plan → approve → run 세 박자가 시작된다.

CMD-01 · REQ-001

2 데이터셋 표 SN-01-2

업무마다 데이터셋·분할·라이선스·내려받기 결과를 한 줄로 보인다. T1 과 T2 는 같은 발화를 쓰고 T3 는 요약 데이터다. 원본 데이터는 저장소에 커밋하지 않는다.

REQ-001 · REQ-002 · POL-15

3 표본 분할 SN-01-3

업무마다 pilot 3건과 main 100건을 고정 시드로 뽑는다. pilot 과 main 은 겹치지 않는다.

REQ-003

4 공통 표본 SN-01-4

main 의 앞 30건이 모든 조합이 함께 푸는 공통 표본이다. 이 중첩 구조 덕에 비싼 모델은 30건만 풀어도 조합끼리 비교할 수 있다.

REQ-004

5 표본 ID 목록 기록 SN-01-5

시드와 표본 ID 목록을 파일에 기록하고 커밋한다. 데이터가 없어도 같은 표본을 다시 뽑을 수 있다.

REQ-005 · POL-15

6 업무 정의 읽음 SN-01-6

업무 YAML 3개(지표·출력 스키마·δ·max_tokens)를 읽었다는 표시. δ 가 비어 있으면 여기서부터 막히고 이후 plan 도 만들어지지 않는다.

REQ-007 · POL-13

7 T2 슬롯 주석 변환 SN-01-7

T2 정답은 원본의 슬롯 주석(annot_utt)을 (종류, 값) 쌍의 JSON 으로 풀어 만든다. 변환이 끝나야 채점기가 쓸 수 있다.

REQ-008

8 라이선스 표시 SN-01-8

데이터셋마다 라이선스를 보인다. T3 의 CC BY-NC-SA 4.0 은 비영리·동일조건이라 리포트에도 같은 표시를 남긴다.

REQ-006 · POL-15

SN-02 plan 결과(배분표·금액 3종)

CLI · 터미널 promptbench

CMD-02 · CMD-04

터미널 — ~/promptbench

```
$ promptbench plan --stage R1
stage          R1 · Haiku 5단계 × main 100건 · 프롬프트 v1 · v2
입력 토큰      count_tokens 로 계산 · 캐시 사용 (요청당 평균 1,200 토큰 가정, 무과금)
```

모델	effort	배분	건수	예상	보수	최악
claude-haiku-5-5	low	100 × 3 업무 × v1·v2	600	$0.0885	$0.1260	$0.8552
claude-haiku-5-5	medium	100 × 3 업무 × v1·v2	600	$0.1110	$0.1560	$0.8552
claude-haiku-5-5	high	100 × 3 업무 × v1·v2	600	$0.1410	$0.2160	$0.8552
claude-haiku-5-5	xhigh	100 × 3 업무 × v1·v2	600	$0.1860	$0.2910	$0.8552
claude-haiku-5-5	max	100 × 3 업무 × v1·v2	600	$0.2460	$0.3960	$0.8552

```
합계           3,000건 · 예상 $0.7725 · 보수 $1.1850 · 최악 $4.2760 (Batch 단가)
예산           누적 $0.1248 / 상한 $5.0000 · 남은 예산 $4.8752
통과 기준      보수 $1.1850 ≤ 남은 예산 $4.8752 ✓ 통과 (최악은 참고)
배치           15개로 분할 (배치당 200건 또는 보수 $0.50 중 먼저 닿는 쪽)
계획           plan-r1-001 · 해시 9b7d1d2f · draft — 과금 호출 0건
출력 형식      structured outputs 스키마 고정 (T3 는 자유 서술)

다음: promptbench approve plan-r1-001

$ promptbench run plan-r1-001 --retry-failed
인프라 실패(errored · expired · canceled) 7건만 담은 새 계획을 만들었습니다 — 실행하지 않았습니다
계획           plan-r1-001-retry · draft · 보수 $0.0025 ≤ 남은 예산 $4.8752 ✓
다음: promptbench approve plan-r1-001-retry   (새 해시이므로 다시 승인)
```

예시 화면 — 수치는 가상 R1 계획 예시. 금액은 빌드할 때 가격표(pricing.yaml)로 계산한 값이며 토큰 수는 가정이다. 아래쪽은 run --retry-failed 가 만든 새 계획이다.

1 명령 입력 SN-02-1

단계마다 plan 으로 계획을 만든다. 계획은 파일로 고정되고 승인 전까지 draft 다. 과금 호출은 0건이다.

CMD-02 · REQ-009

2 stage 와 배분 요약 SN-02-2

stage 는 pilot · R1 · judge-R1 · R2 · judge-R2 · latency 다. R1 은 Haiku 5단계 × main 100건 × 프롬프트 v1 · v2 다. R2 는 R1 에서 이긴 프롬프트 하나로 Sonnet low/medium · Opus low 를 공통 30건에 돌리고 R1 · judge-R1 이 끝나지(done) 않으면 만들어지지 않는다. judge-R1 · judge-R2 는 T3 공통 30건 요약의 채점 요청을 Sonnet 5.5 effort high 로 보낸다.

REQ-009 · REQ-010 · REQ-011 · REQ-012 · REQ-042 · POL-22

3 입력 토큰 계산 SN-02-3

입력 토큰은 count_tokens 로 (모델, 프롬프트 버전, 항목)별로 세고 캐시한다. 무료라 승인 전에 써도 된다. count_tokens 는 출력 스키마와 effort 설정을 받는다(구현 1단계에서 확인함).

REQ-013 · POL-05

4 배분표 SN-02-4

모델 × effort 마다 건수와 금액 3종을 보인다. 건수는 업무 3개 × 프롬프트 2개 × 표본 수다. 비용은 입력·출력 토큰에 단가를 곱하고 Batch 는 50% 를 깎는 식으로 계산한다(예시). 공통 30건은 줄이지 않는다.

REQ-004 · REQ-020 · POL-02

5 금액 3종 합계 SN-02-5

예상은 출력 토큰 평균, 보수는 p90, 최악은 max_tokens 전부다. 출력 통계는 파일럿 결과로 낸 값이다. 단가는 config/pricing.yaml 한 곳에서 읽는다. 모든 금액은 $0.0000 표기이고 예시다.

REQ-014 · REQ-018 · REQ-019 · POL-05 · POL-06 · POL-18

6 누적과 남은 예산 SN-02-6

상한 $5 는 단계마다가 아니라 프로젝트 전체 누적 장부에 적용한다. 남은 예산 = 상한 − 누적 사용액이다.

POL-01 · REQ-014

7 통과 기준 SN-02-7

통과 기준은 보수 ≤ 남은 예산이다(파일럿은 출력 통계가 없어 최악 ≤ 남은 예산). 최악은 참고 값이라 남은 예산을 넘어도 보수가 안이면 통과다(이 예시는 최악도 안에 든다). 넘으면 계획기가 줄이고, 줄여도 안 되면 SN-04 처럼 계획을 만들지 않는다.

REQ-014 · REQ-015 · POL-21

8 배치 분할 SN-02-8

SN-03 approve 승인 화면 CLI · 터미널 promptbench

터미널 — ~/promptbench

```
1 $ promptbench approve plan-r1-001
2 stage          R1 · 계획 plan-r1-001 · 해시 9b7d1d2f draft
  금액은 예시 — 단가는 가격표로 계산하고 토큰 수는 가정값입니다

3 모델                          effort    프롬프트    건수    보수 금액
  claude-haiku-5-5              low       v1 · v2     600     $0.1260
  claude-haiku-5-5              medium    v1 · v2     600     $0.1560
  claude-haiku-5-5              high      v1 · v2     600     $0.2160
  claude-haiku-5-5              xhigh     v1 · v2     600     $0.2910
  claude-haiku-5-5              max       v1 · v2     600     $0.3960

4 예상 금액        $0.7725  출력 토큰 평균 기준
  보수 금액        $1.1850  출력 토큰 p90 기준 — 통과 기준
  최악 금액        $4.2760  max_tokens 전부 쓴다고 가정
5 누적 사용액      $0.1248 / 상한 $5.0000 · 남은 예산 $4.8752 → 실행 뒤 최대 $1.3098 (보수 기준)
6 가격표 확인일    2026-10-20 (오늘 2026-11-23 — 34일 전)
  # 확인일 30일 초과 시에만 (이 예는 34일)

7 ⚠ 경고 — 가격표 확인일이 30일을 넘었습니다 (34일 전)
    이유: 단가가 바뀌었다면 위 금액이 실제와 다를 수 있습니다
    다음: config/pricing.yaml 을 공식 가격 페이지와 맞춘 뒤 다시 계획하거나, 확인하고 승인하세요

8 # (latency 단계일 때만) Batch 할인 없음 — 동기 호출이라 단가에 50% 할인이 없습니다

9 이 계획을 승인합니다 — 승인은 해시 9b7d1d2f 에 묶입니다. 계획이 바뀌면 다시 승인해야 합니다. 진행할까요? [y/N] _
```

예시 화면 — 수치는 가상 승인 화면 예시. 금액은 pricing.yaml 로 계산한 예시 값이다. 가격표 확인일 경고와 latency 줄은 해당 조건에서만 나온다.

1 명령 입력 SN-03-1

승인은 plan_id 로 한다. 과금되는 모든 단계(pilot · R1 · judge-R1 · R2 · judge-R2 · latency)가 같은 승인 화면을 거친다. 승인 전에는 과금 호출 0건이다. API 키는 환경변수 ANTHROPIC_API_KEY 하나만 쓰며 없으면 과금 호출 전에 알리고 멈춘다.

CMD-03 · REQ-021 · REQ-022 · REQ-024 · REQ-055 · POL-03 · POL-16

2 stage 와 해시 SN-03-2

stage · 계획 ID · 해시 앞 8자. 승인은 이 해시에 묶인다. 계획 내용이 바뀌면 해시가 바뀌어 승인이 무효가 된다. 상태 draft 는 ST-P-01 이다.

REQ-021 · REQ-023 · POL-04

3 조합표 SN-03-3

모델 · effort · 프롬프트 · 건수 · 보수 금액을 조합마다 보인다. 승인하는 사람이 무엇을 몇 건 돌리는지 먼저 본다.

REQ-021

4 금액 3종 SN-03-4

예상(출력 평균) · 보수(출력 p90) · 최악(max_tokens 전부)을 모두 보인다. 승인의 기준은 보수다. 모두 $0.0000 표기의 예시다.

REQ-021 · POL-05 · POL-21

5 누적 사용액과 상한 SN-03-5

지금까지 쓴 누적 사용액, 상한 $5, 남은 예산, 보수 기준으로 실행 뒤 최대 누적을 한 줄로 보인다.

REQ-021 · POL-01

6 가격표 확인일 SN-03-6

pricing.yaml 의 확인일을 항상 보인다. 단가가 오래되면 금액을 믿기 어렵기 때문이다.

REQ-025 · POL-19

7 확인일 경고 SN-03-7

확인일로부터 30일을 넘기면(31일째부터) 경고 줄이 붙는다. 경고는 차단하지 않는다 — 확인하고 승인할 수 있다. 30일 이하면 이 줄은 없다. 목업은 조건부임을 줄 위 주석으로 표시했다.

REQ-025 · POL-19

8 Batch 할인 없음 SN-03-8

stage latency 계획은 동기 호출이라 Batch 50% 할인이 없다. 승인 화면에 이 줄을 더해 금액이 두 배로 보이는 이유를 알린다. 그 밖의 stage 에는 나오지 않는다. 목업에서는 흐리게 보여 조건부임을 나타낸다.

REQ-026

9 승인 입력 [y/N] SN-03-9

SN-04 예산 초과로 계획 생성 실패 CLI · 터미널 promptbench

CMD-02

터미널 — ~/promptbench

1 `$ promptbench plan --stage R1`

예산 누적 $4.7511 / 상한 $5.0000 · 남은 예산 **$0.2489**

보수 금액이 남은 예산을 넘어 줄여 봅니다 (공통 30건은 줄이지 않음)

2

단계	줄인 것	건수	보수 금액	판정
0	전체 계획	3,000	$1.1850	> $0.2489
1	Haiku 비공통 표본 줄임(공통 30건만)	900	$0.3555	> $0.2489

3

✗ **계획을 만들지 않았습니다 — 보수 금액 $0.3555 가 남은 예산 $0.2489 를 넘습니다**

이유: 줄일 수 있는 것을 모두 줄여도 공통 30건만으로 남은 예산을 넘습니다 (상한 $5.0000 는 단계마다가 아니라 전체 합계)

다음: promptbench status 로 장부를 확인하고, 남은 예산에 맞는 조합으로 범위를 정한 뒤 다시 plan 하세요

4 계획 파일 없음 · 승인 없음 · 과금 호출 0건 · 종료코드는 실패(0 아님)

예시 화면 — 수치는 가상 줄일 수 있는 것을 모두 줄여도 남은 예산을 넘는 경우. 앞선 지출이 커서 남은 예산이 적다고 가정한 예시다.

1 **명령 입력** SN-04-1

plan 은 만들 때마다 누적 사용액을 읽어 남은 예산을 다시 계산한다.

CMD-02 · POL-01

2 **줄이는 순서 표** SN-04-2

R1 에서 계획기가 줄일 수 있는 것은 Haiku 비공통 표본(31~100번째) 하나다 — 줄인 뒤 보수 금액을 다시 본다. 공통 30건은 줄이지 않는다. 표의 금액은 가격표로 계산한 예시다.

REQ-015 · POL-21 · POL-01

3 **오류 3줄** SN-04-3

무엇이(계획을 만들지 않음) / 왜(공통 30건만으로도 넘음) / 다음에 할 일(status 로 장부 확인 후 범위 조정)의 세 줄 형식이다. 상한을 올리라는 안내는 하지 않는다.

REQ-016 · REQ-054 · POL-21 · POL-01

4 **부작용 없음** SN-04-4

계획 파일도 승인도 만들어지지 않고 과금 호출은 0건이다. 종료코드는 실패(0 아님)로 끝난다는 것은 REQ-016 의 수용 기준이다.

REQ-016 · POL-03

SN-05 run 진행 표시

CLI · 터미널 promptbench



예시 화면 — 수치는 가상 승인된 계획을 작은 배치로 나눠 순서대로 낸다. 배치가 끝날 때마다 실제 비용을 장부에 더한다. 실제 비용은 평균 출력 토큰이 가정의 1.1배라고 가정한 예시다.

1 명령 입력 SN-05-1

run 은 승인된(ST-P-02) 계획만 실행한다. 과금 호출을 할 수 있는 유일한 명령이다.

CMD-04 · POL-03

2 승인 해시 확인 SN-05-2

승인된 해시와 계획 파일의 해시가 같을 때만 running 으로 넘어간다. 다르면 제출하지 않는다.

POL-04 · POL-03

3 배치 표 SN-05-3

배치마다 요청 수, 보수 금액, 상태, 실제 비용, 누적을 보인다. 보수는 파일럿에서 잰 출력 토큰 통계로 계산한다. 배치는 한 번에 하나씩 순서대로 낸다. 비용은 결과의 usage 에 단가를 곱한 값이다.

REQ-027 · REQ-018 · REQ-020 · POL-07 · POL-20 · POL-02

4 진행 줄 SN-05-4

끝난 배치 수와 건수, 지금 기다리는 배치, 남은 건수를 한 줄로 보인다. 배치 결과는 오래 걸릴 수 있어 기다리는 동안 이 줄이 갱신된다.

REQ-035

5 장부 줄 SN-05-5

누적 사용액과 상한, 남은 예산, 다음 배치의 보수 금액을 같은 줄에 두어 여유를 바로 본다. 배치 하나가 끝날 때마다 장부에 한 줄이 더해진다.

REQ-028 · POL-02 · POL-07

6 결과 처리 SN-05-6

결과는 순서가 아니라 custom_id 로 요청과 연결한다. errored · expired · canceled 는 오답으로 센다. 일시 오류는 SDK 의 재시도에만 맡긴다. 결과 usage 에 thinking 토큰이 출력으로 잡힌다(구현 1단계에서 확인함, 근사).

REQ-032 · REQ-033 · REQ-034 · POL-09

7 속도 측정 SN-05-7

TTFT 와 총 시간은 Batch 결과로 잴 수 없어 stage latency 의 동기 호출로 따로 잰다.

REQ-036

8 결과 저장 SN-05-8

원본 응답은 JSONL 과 SQLite 에 저장해 나중에 다시 채점할 수 있다. API 키 값은 어디에도 기록하지 않는다.

REQ-037 · POL-17

SN-06 run 상한 정지(halted) CLI · 터미널 promptbench

CMD-04

터미널 — ~/promptbench

```
1 $ promptbench run plan-r1-001
  진행            12/15 배치 끝 · 2,400건 · 배치 13 제출 전 점검
```

2

항목	금액
누적 사용액	$4.9208
남은 예산 (상한 $5.0000 − 누적)	$0.0792
다음 배치 보수 금액 (200건)	$0.1320

3

```
  ⚠ 상한 직전에 멈췄습니다 — 배치 13 의 보수 금액 $0.1320 가 남은 예산 $0.0792 를 넘습니다
    이유: 제출하면 상한 $5.0000 를 넘을 수 있어 제출 전에 정지했습니다 (이 시점에 과금 호출 없음)
    다음: promptbench status 로 확인하세요. 상한은 올리지 않으며, 남은 요청은 새 계획으로 줄여 다시 승인받아야
    합니다
```

```
4 계획 상태       plan-r1-001 halted
5 보존            끝난 배치 12개의 결과는 집계에 사용 · 남은 600건은 pending 으로 남김 (삭제 안 함)
  종료코드 2(D06 설계 결정) · 배치 ID 와 장부는 DB 에 기록됨
```

예시 화면 — 수치는 가상 다음 배치의 보수 금액이 남은 예산보다 크면 제출 전에 멈춘다. 지출은 상한 직전이라고 가정한 예시이며 SN-05 의 장부를 이어 가지 않는다.

1 **명령 입력** SN-06-1
run 은 배치를 내기 전마다 누적 사용액을 읽는다.
CMD-04 · POL-07

2 **제출 전 점검표** SN-06-2
누적 사용액, 남은 예산, 다음 배치의 보수 금액을 보인다. 보수 금액이 남은 예산보다 크면 붉게 표시한다. 금액은 예시다.
REQ-029 · POL-07

3 **경고 3줄** SN-06-3
무엇이(상한 직전에 멈춤) / 왜(제출하면 상한을 넘을 수 있음) / 다음에 할 일 세 줄이다. 정지 시점에 과금 호출은 하지 않는다. 상한을 올리지 않고 새 계획으로 줄여 다시 승인받는다.
REQ-029 · POL-07 · POL-08

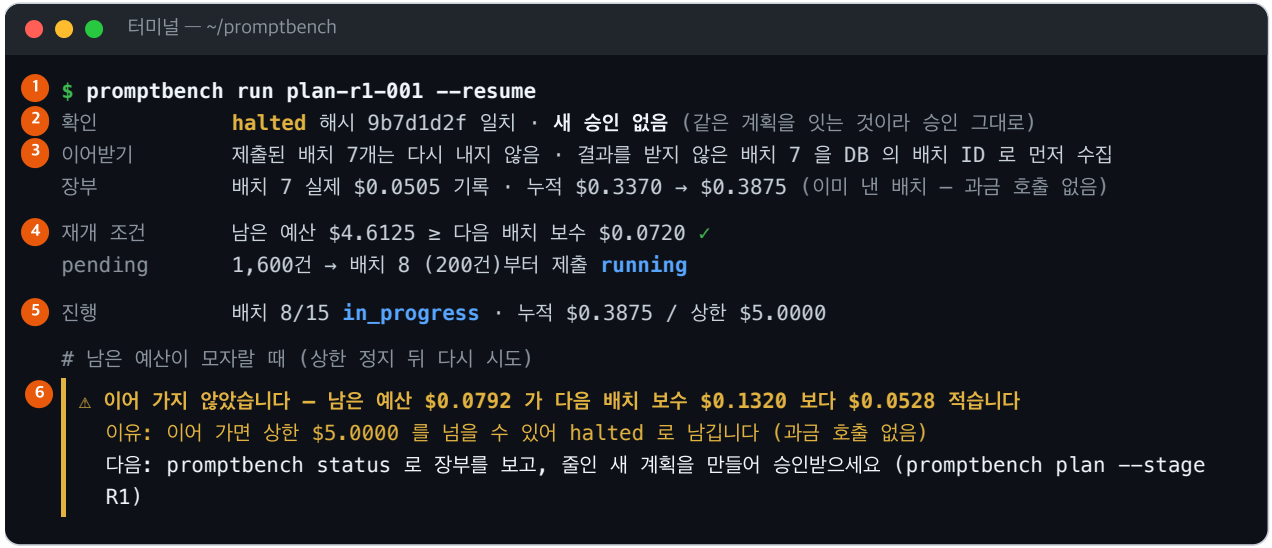
4 **계획 상태 halted** SN-06-4
계획은 halted(ST-P-05)가 된다. 상한 정지와 프로세스 중단이 같은 상태다.
REQ-029 · POL-08

5 **보존** SN-06-5
끝난 배치의 결과는 집계에 쓰고, 아직 내지 않은 요청은 삭제하지 않고 pending 으로 남긴다.
POL-08

SN-07 run --resume

CLI · 터미널 promptbench

CMD-04



예시 화면 — 수치는 가상 중단된 계획을 같은 해시로 잇는다. 위쪽은 재개 성공, 아래쪽은 남은 예산이 모자라 halted 로 남는 경우다.

1 명령 입력 SN-07-1

--resume 은 halted 계획, 또는 running 인데 run 시작 때 기록한 소유자(잠금 · pid)가 살아 있지 않은 계획(먼저 halted 로 바꾼다)을 이어 간다. 소유자가 살아 있으면 거부한다. 재시도가 아니며 새 계획이 아니다.

CMD-04 · REQ-030 · POL-08

2 승인 확인 SN-07-2

같은 계획 해시이므로 새 승인은 필요 없다. 해시가 다르면 다른 계획이라 승인부터 다시 받아야 한다.

POL-04 · POL-08

3 이어받기 SN-07-3

이미 낸 배치는 다시 내지 않는다. 제출됐지만 결과를 받지 않은 배치를 DB 에 기록된 배치 ID 로 먼저 조회 · 수집하고 실제 비용을 장부에 더한다 — 이미 낸 배치라 과금 호출이 아니다. 같은 요청이 두 번 과금되지 않는다.

REQ-030 · POL-08

4 재개 조건 SN-07-4

수집 · 기록 뒤의 누적으로 남은 예산이 다음 배치의 보수 금액 이상일 때만 이어 간다. 수집이 먼저라야 아직 장부에 없던 비용까지 넣고 판정한다. 금액은 예시다.

POL-08

5 진행 줄 SN-07-5

재개 뒤 진행 표시는 SN-05 와 같다. 누적은 중단 전부터 이어진다.

REQ-035

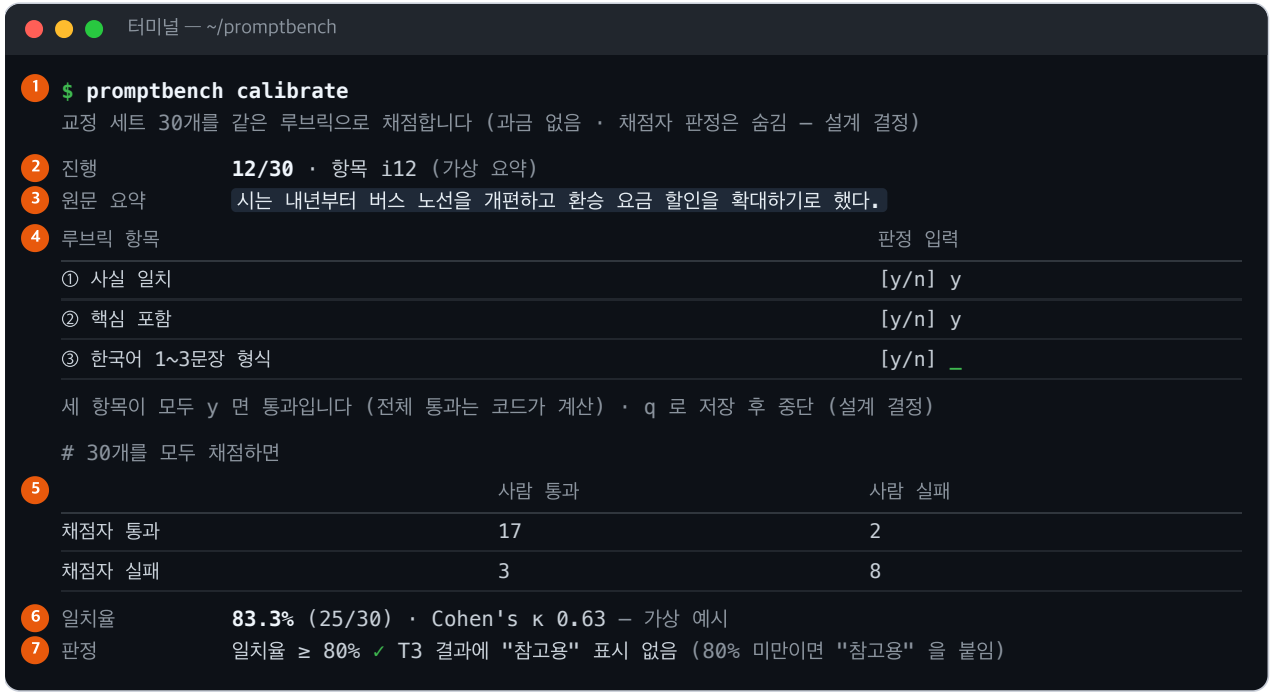
6 재개 거부 경고 SN-07-6

남은 예산이 모자라면 이어 가지 않고 halted 로 남긴다. 무엇이 / 왜 / 다음에 할 일 세 줄이다. 이 거부 화면 자체는 설계 결정(D06에서 정함)이며 조건은 POL-08 의 재개 조건을 따른다.

POL-08 · POL-07

SN-08 calibrate 채점 화면

CLI · 터미널 promptbench



예시 화면 — 수치는 가상 사람이 교정 세트 30개를 같은 루브릭으로 채점한다. 요약문과 교차표는 가상이며 일치율과 κ 는 교차표에서 계산한 값이다.

1 명령 입력 SN-08-1

calibrate 는 계획도 승인도 없고 과금도 없다. judge-R2 계획이 끝난(ST-P-04) 뒤 사람이 한다 — judge-R1 은 R2 의 전제라 이미 끝나 있다.

CMD-06 · REQ-043

2 진행 표시 SN-08-2

현재 항목 번호와 전체 30개 중 위치를 보인다. 채점자의 판정은 숨겨 사람의 판단이 기울지 않게 한다. 설계 결정(D06에서 정함): 앞 문서에 근거가 없다.

REQ-043

3 요약문 SN-08-3

채점할 요약을 인용 상자 안에 보인다. 채점 대상 글은 데이터이며 지시로 읽지 않는다.

REQ-043 · POL-12

4 루브릭 입력 SN-08-4

루브릭 세 항목(사실 일치 · 핵심 포함 · 한국어 1~3문장 형식)에 y/n 으로 답한다. 세 항목이 모두 통과여야 그 요약이 통과다. q 로 저장 후 중단하는 입력은 설계 결정(D06에서 정함).

REQ-043 · POL-12 · POL-22

5 교차표 SN-08-5

30개를 마치면 채점자와 사람의 통과/실패 교차표가 나온다. 숫자는 가상 예시다.

REQ-043

6 일치율과 κ SN-08-6

일치율과 Cohen's κ 는 어느 경우에나 리포트에 적는다. 교차표에서 계산한 값이다.

REQ-043 · REQ-044 · POL-12

7 참고용 판정 SN-08-7

일치율이 80% 미만이면 T3 결과에 참고용 표시를 붙이고 루브릭 개선을 한계로 남긴다. 80% 이상이면 표시하지 않는다.

REQ-044 · POL-12

SN-09 report 요약 출력

CLI · 터미널 promptbench

CMD-07

터미널 — ~/promptbench

1 `$ promptbench report`

리포트 → report.md · 파레토 그림 · Haiku effort 곡선 (과금 없음 · 장부와 DB 를 읽기만 함)

T1 문의 분류 — 정확도 · 공통 30건 가상 예시

2

조합	정답	정확도	1,000건당	(B–C) 상한	동급	결과
claude-opus-5-5/low	28/30	0.933	$9.0000	—	동급	B · 기본값
claude-sonnet-5-5/medium	28/30	0.933	$4.5000	0.1000	아님	
claude-haiku-5-5/high	27/30	0.900	$0.2250	0.1000	아님	
claude-haiku-5-5/low	24/30	0.800	$0.2250	0.2667	아님	

3 기본값 **claude-opus-5-5/low** — 공통 30건 기준 판정, 확정에는 추가 표본 필요 (δ 0.03 · 부트스트랩 10,000회 · 고정 시드)

4 실패 유형 오답 집계 — 형식 오류 0건 · 거절 0건 · max_tokens 잘림 0건 · errored 0건 · expired 0건 · canceled 0건 (0건도 칸을 유지)

5 T3 채점 불가 채점자 쪽 실패는 후보의 오답이 아니라 **채점 불가** 로 따로 세고 분모에서 제외

6 신뢰도 일치율 83.3% · κ 0.63 · 80% 이상이라 "참고용" 표시 없음 · 채점자(Sonnet)와 같은 계열 후보는 자기 선호 편향 가능

7

비용	예상	보수	실제	예상 대비
R1 (예시 · 실제 = 평균 출력의 1.1배 가정)	$0.7725	$1.1850	$0.8317	+7.7%

8

✗ **T3 기본값 선정을 거부했습니다 — δ 가 계획 때와 다릅니다**

이유: 계획 해시에 기록된 δ 0.05 ≠ 업무 YAML 의 현재 값 0.04 (δ 는 실행 전에 고정)

다음: δ 를 0.05 로 되돌리거나, 새 값으로 새 계획을 만들어 처음부터 실행하세요

예시 화면 — 수치는 가상 report 가 터미널에 요약하는 출력. 결과표는 D03 의 가상 예시(고정 시드 30건)를 같은 함수로 다시 실행한 값이며 실측이 아니다. Sonnet · Opus 행은 R1 이 아니라 R2(Sonnet low/medium · Opus low × 공통 30건, 이긴 프롬프트)에서 온 것으로 가정한다.

1 명령 입력 SN-09-1

report 는 장부와 DB 를 읽어 마크다운 리포트(결과표 · 데이터셋 라이선스 · 한계)와 품질 대 비용 파레토 그림 · Haiku effort 곡선을 만든다. 과금은 없다.

CMD-07 · REQ-047 · REQ-048 · REQ-053 · REQ-006

2 업무별 결과표 SN-09-2

업무마다 지표(T1 정확도 · T2 필드 F1 · T3 루브릭 통과율)와 비용을 조합별로 보이고 최고 조합 B 와의 차이 상한으로 동급을 표시한다. 표본이 30건이라 상한이 넓다. 예시의 Sonnet · Opus 행은 R1 이 아니라 R2(Sonnet low/medium · Opus low × 공통 30건, 이긴 프롬프트) 결과다.

REQ-039 · REQ-040 · REQ-046 · REQ-050 · POL-11 · POL-14

3 기본값 선정 SN-09-3

선정 규칙을 실행한 결과다. 30건 기준 판정이라 확정에는 추가 표본이 필요하다는 문구를 붙인다. δ 와 부트스트랩은 실행 전에 사전 등록한 값이다.

REQ-050 · REQ-051 · REQ-052 · POL-13 · POL-14

4 실패 유형 건수 SN-09-4

후보 쪽 실패(형식 오류 · 거절 · 잘림 · 배치 실패)는 오답으로 세고 유형별 건수를 따로 보고한다. 0건인 유형도 칸을 유지한다.

REQ-041 · POL-09

5 T3 채점 불가 SN-09-5

T3 에서 채점자 쪽 실패는 후보의 오답이 아니라 채점 불가로 따로 세고 통과율 분모에서 뺀다. 후보 쪽 실패는 분모에 남는다.

REQ-040 · POL-09 · POL-22

6 채점 신뢰도 SN-09-6

교정 일치율과 κ 를 적고, 일치율이 낮으면 참고용 표시가 붙는다. 채점자(Sonnet 5.5)와 같은 계열인 Sonnet 후보에 대한 자기 선호 편향 가능성도 한계로 적는다.

REQ-044 · REQ-045 · POL-12

7 예상 대 실제 비용 SN-09-7

계획 당시의 예상 · 보수 금액과 장부의 실제 비용을 나란히 적고 오차를 보고한다. 오차가 커도 규칙은 바뀌지 않고 다음 계획의 입력으로만 쓴다. 금액은 예시이고 실제 비용은 평균 출력의 1.1배라고 가정한 값이다.

REQ-049 · POL-06

8 δ 변경 거부 SN-09-8

실행 시작 뒤 δ 가 계획 때 값과 달라졌으면 기본값 선정을 거부한다. 무엇이 / 왜 / 다음에 할 일 세 줄 형식이다. δ 0.04 는 거부를 보이려는 가정이다.

REQ-051 · POL-13

SN-10 status 장부

CLI · 터미널 promptbench



예시 화면 — 수치는 가상 status 는 장부와 진행 상태를 읽기만 한다. 값은 SN-05 · SN-07 과 같은 가정에서 계산한 예시다.

1 명령 입력 SN-10-1

status 는 읽기만 하며 과금 호출도 승인도 필요 없다. 언제든 실행해도 된다.

CMD-05 · REQ-038

2 예산 줄 SN-10-2

프로젝트 전체 누적 사용액, 상한 $5, 남은 예산을 한 줄로 보인다. 단계마다가 아니라 합계다.

REQ-054 · REQ-038 · POL-01

3 stage 별 표 SN-10-3

단계마다 계획 · 상태 · 배치 진행 · 실제 비용을 보인다. plan-pilot-001 같은 계획 ID 모양은 설계 결정(D06에서 정함)의 예시다. 상태 이름은 states.yaml 의 계획 상태 그대로다.

REQ-038 · REQ-028

4 실행 중 단계 진행 SN-10-4

끝난 건수와 pending 건수, 다음 배치 보수 금액을 보인다. 이어 하려면 --resume 이다.

REQ-038

5 마지막 장부 SN-10-5

배치마다 한 줄(실제 비용 · 누적 합계)이 쌓인다. 장부의 금액은 예상이 아니라 usage 로 다시 계산한 실제 비용이다. 제출이 실패하면 기록하지 않는다.

REQ-028 · POL-02

6 가격표 확인일 SN-10-6

pricing.yaml 의 확인일을 보인다. 30일을 넘기면 경고를 붙인다.

POL-18 · POL-19